



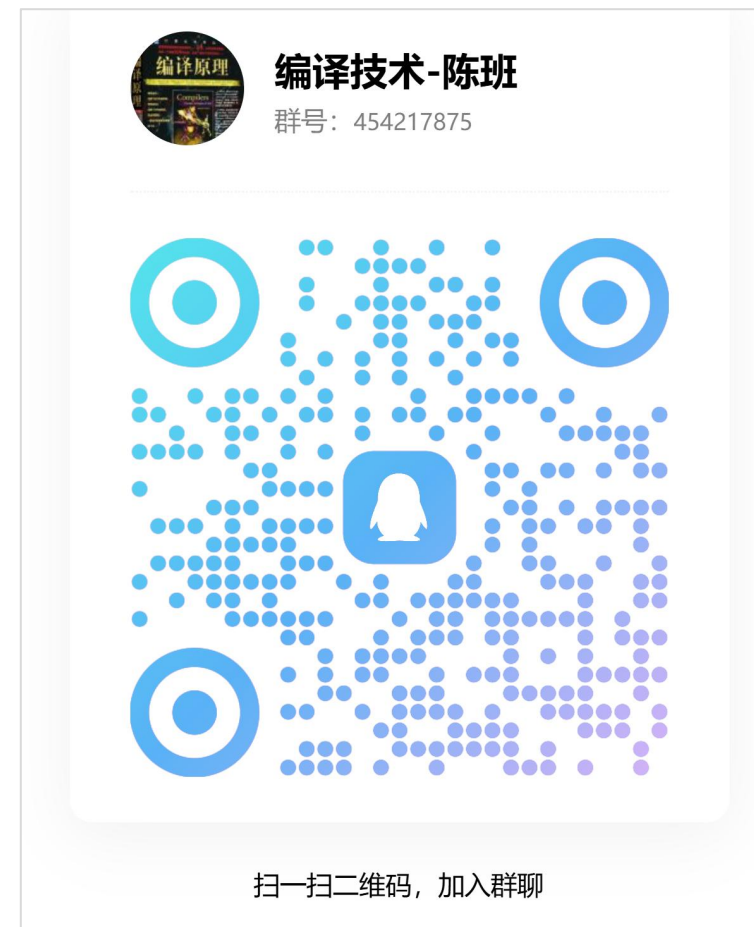
信息与软件工程学院

编译技术

主讲教师： 陈安龙

自我介绍

- 姓 名： 陈安龙
- 主讲课程： 软件类相关课程
- 电 话： 13688434459
- E-Mail: chenanlong@foxmail.com
- QQ课程群： 454217875



课程的性质与任务

- 编译技术课程通常属于**计算机科学或软件工程专业的高级课程**，旨在给学生讲解**如何设计、实现和优化编译器的基本知识**。
- 该课程涵盖了编程语言的**词法分析、语法分析、语义分析、中间代码生成、代码优化和目标代码生成**等内容。
- 在学习编译技术课程时，可以深入了解**编程语言的内部工作原理**，掌握**编译器的设计和实现方法**，以及**优化编译器性能的技术**。

课程的特点和目标

- 该课程的性质是**理论性和实践性相结合的**，既需要**理解编译原理的基本概念和算法**，又需要通过**实际编写编译器或相关项目**来加深对知识的理解和掌握。
- 通过学习编译技术或编译原理，可以**提高对编程语言和计算机系统的理解**，培养编写**高效、可靠编译器的能力**，为日后从事软件开发、系统设计等工作打下坚实的基础。

与编译技术相关的工作

- **AI编译器工程师**：为深度学习模型编写编译器，将PyTorch、MindSpore等框架的模型自动翻译并优化到GPU、NPU等硬件上。
- **编译器后端工程师**：这类岗位聚焦于把IR生成目标机器码，是编译器最底层的环节。
日常职责：指令选择、寄存器分配、指令调度、重写层、ABI适配等。
- **编译器前端与基础设施工程师**：这个方向偏重于语言与编译器的前端设计，也包括基础编译框架的开发。
- **性能优化工程师**：不一定要完整开发编译器，但会使用编译技术与分析工具来优化实际应用。
- **运行时与JIT引擎工程师**：编译和运行时往往密不可分，JIT编译是运行时的重要部分。
开发解释器、JIT编译器、GC，优化运行时执行路径。
- **芯片软件工具链工程师**：面向硬件公司，负责为特定芯片提供完整的编译工具链和SDK。把编译后端适配到自研芯片，提供算子库、调试工具、性能分析工具。

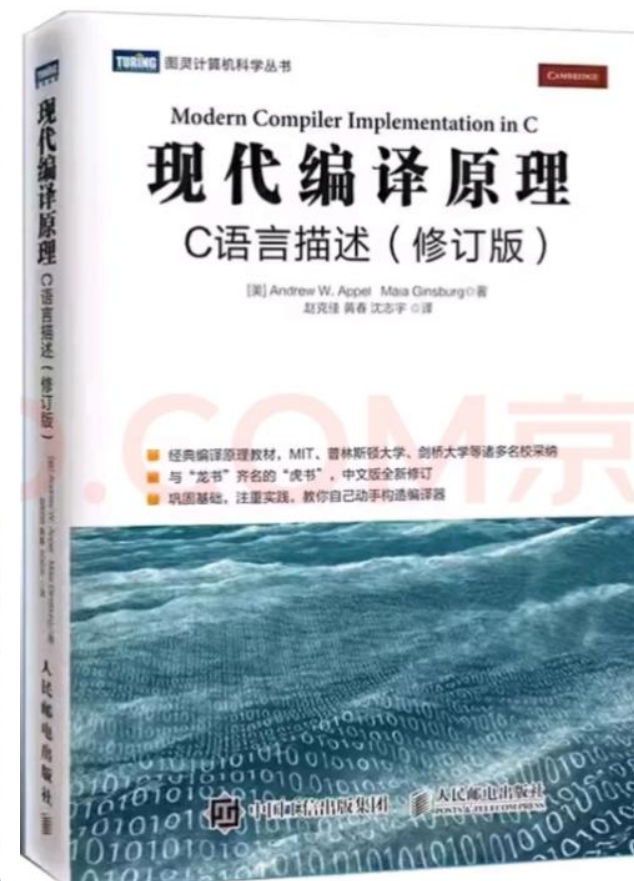
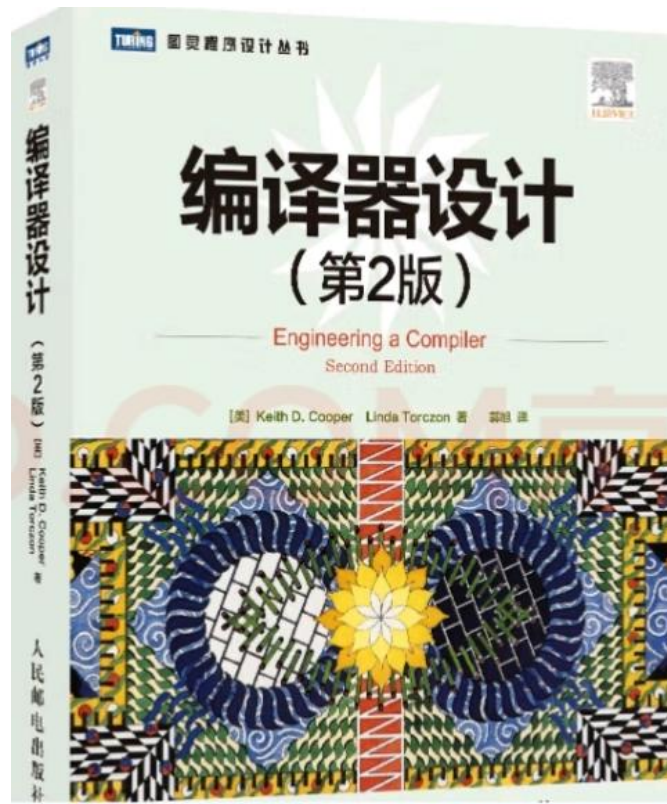
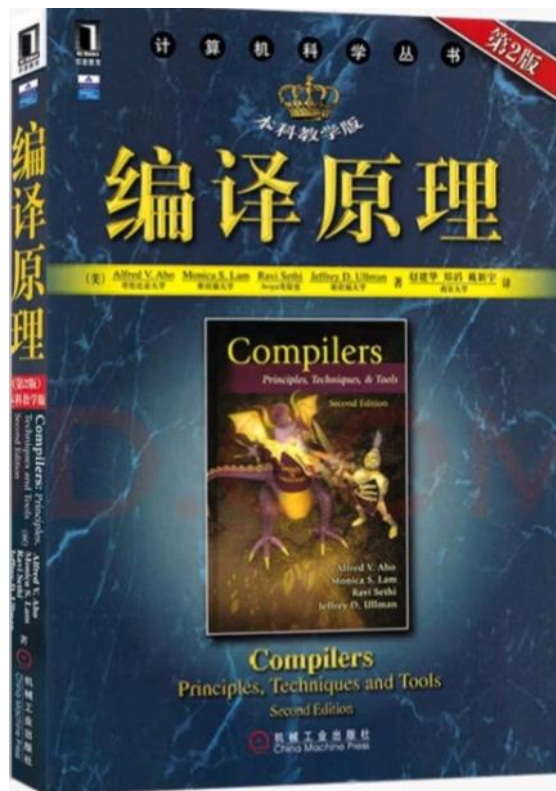
与编译器开发相关的工具

- **Flex 和 Bison:** Flex 用于生成词法分析器, Bison用于生成语法分析器。它们可以帮助开发人员快速构建编译器的前端部分。
- **LLVM:** LLVM 是一个开源的编译器基础软件, 提供了丰富的编译器组件和工具, 如中间表示(IR)、代码优化器、目标代码生成器等。开发人员可以使用 LLVM 来构建高性能的编译器。
- **GCC:** GCC 是一个流行的开源编译器套件, 支持多种编程语言和平台。开发人员可以使用 GCC 来开发和测试编译器, 学习编译器的设计和实现。
- **Clang:** Clang 是基于 LLVM 的 C/C++ 编译器前端, 提供了高性能和可扩展的编译器前端实现。开发人员可以使用 Clang 来构建自定义的编译器前端。
- **IDE (集成开发环境):** 开发编译器时, 可以选择一些功能强大的集成开发环境, 如Visual Studio、Eclipse、IntelliJ IDEA 等, 来编写、调试和测试编译器代码。

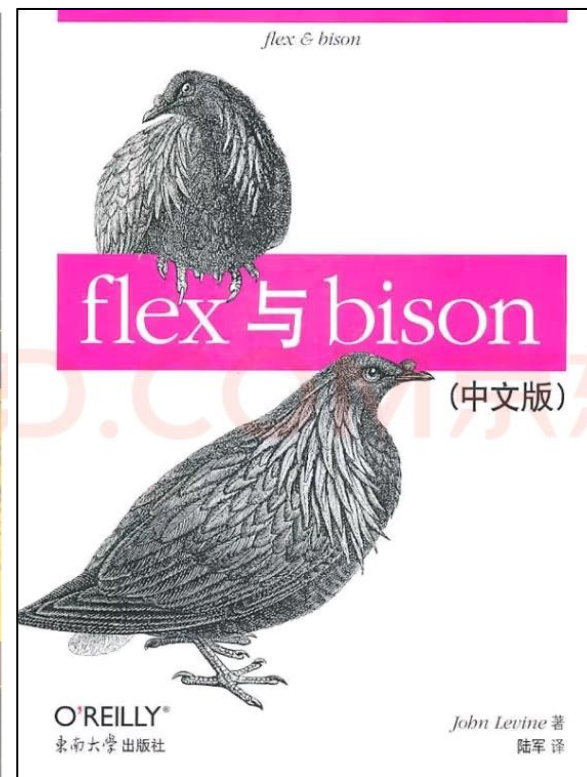
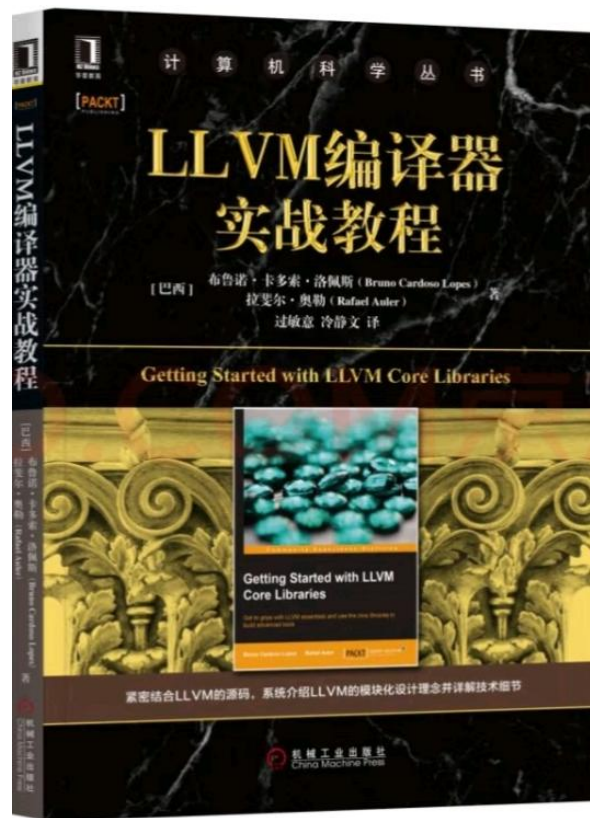
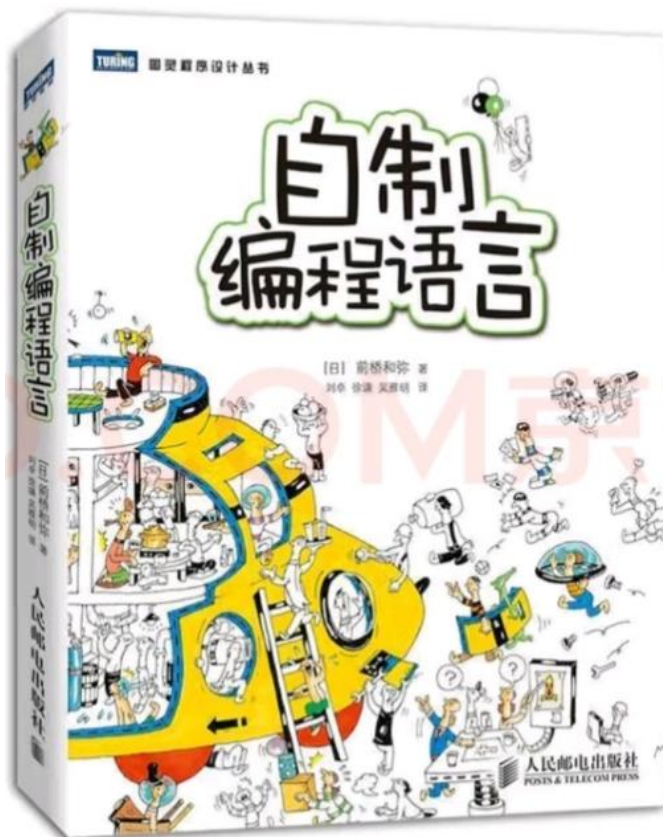
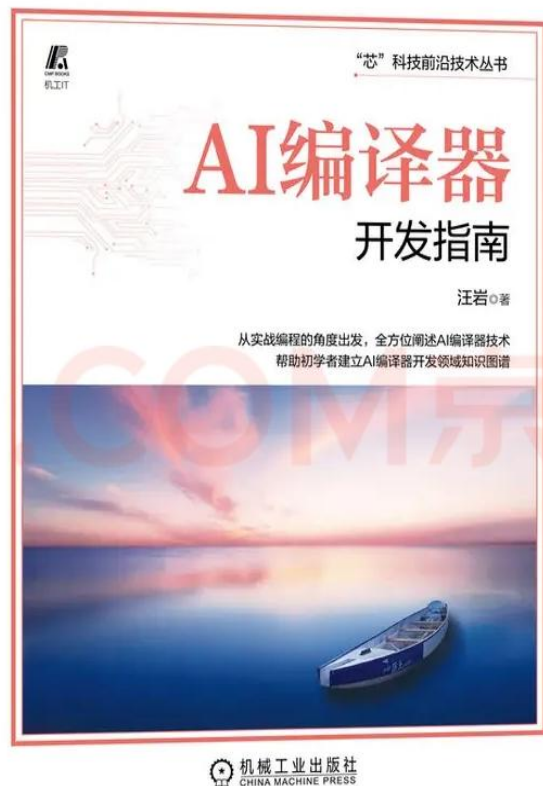
网络学习资料

- <https://llvm.org/docs/LangRef.html>
- <https://llvm-tutorial-cn.readthedocs.io/en/latest/index.html>
- <https://kaleidoscope-llvm-tutorial-zh-cn.readthedocs.io/zh-cn/latest/>
- <https://llvm.org/>
- <https://releases.llvm.org/>
- <https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/index.html>

课程教材及参考书



课程教材及参考书



该课程的考核与评价

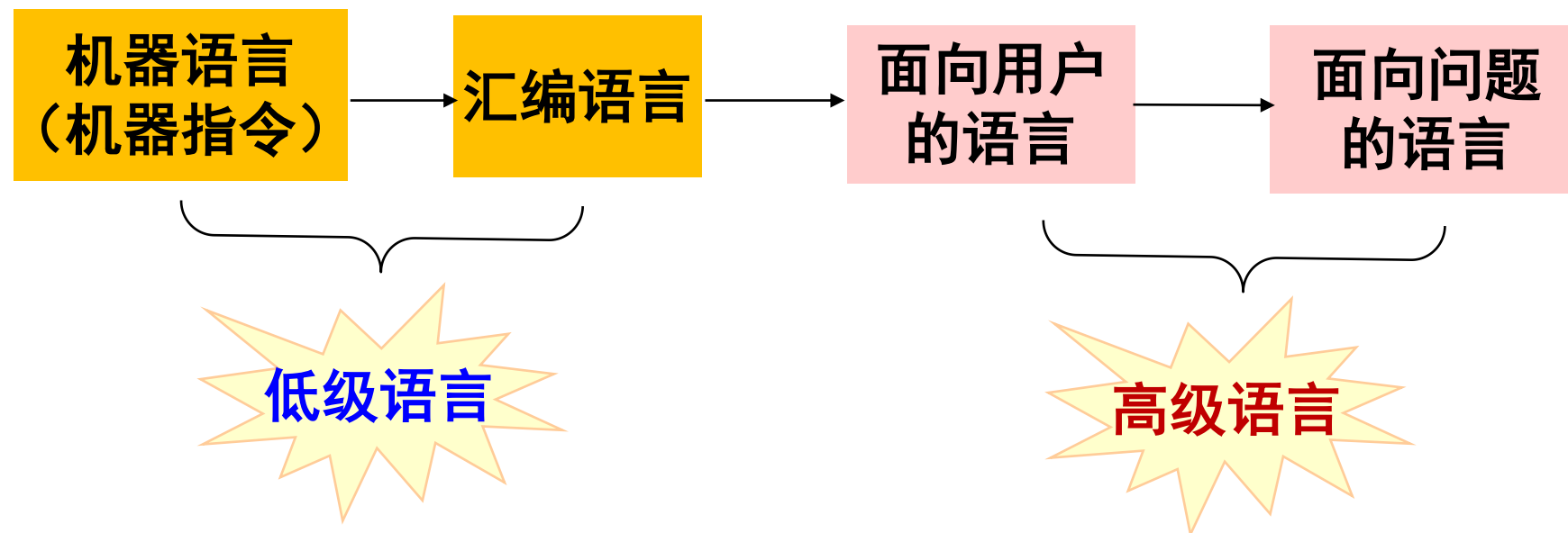
■ 考核方式：

- ① 课程考核评价：平时成绩和期末考试
- ② 综合评价成绩 = （平时成绩：课堂作业 + 四次实验）×50% + 期末考试×50%
- ③ 如果期末笔试成绩低于规定分数时，平时成绩打折处理后，按照成绩构成比例计算最终成绩。具体打折规则等候课程组讨论后通知大家，请等候关注！

第1章 引论

- 计算机语言的简介
- 编译程序所处的层次
- 编译器的特点及功能
- 编译器的内部结构
- 编译器的构造工具
- AI 编译器简介

1. 计算机语言的发展



机器语言(Machine Language)

0、1代码与助记符：更接近于计算机硬件指令系统的工作

1011 1000 0010 1011 0001 0101 (B82B15)

1000 1110 1101 1000 (8ED8)

1010 0001 0000 0000 0000 0000 (A10000)

1000 1011 0001 1110 0000 0010 0000 0000 (8B1E0200)

1011 1001 0000 0000 0000 0000 (B90000)

0000 0011 1100 1000 (03C8)

0000 0011 1100 1011 (03CB)

1000 1011 0000 1110 0000 0100 0000 0000 (8B0E0400)

1011 1000 0000 0000 0100 1100 (B8004C)

1100 1101 0010 0001 (CD21)

■ 汇编语言(Assemble Languag)

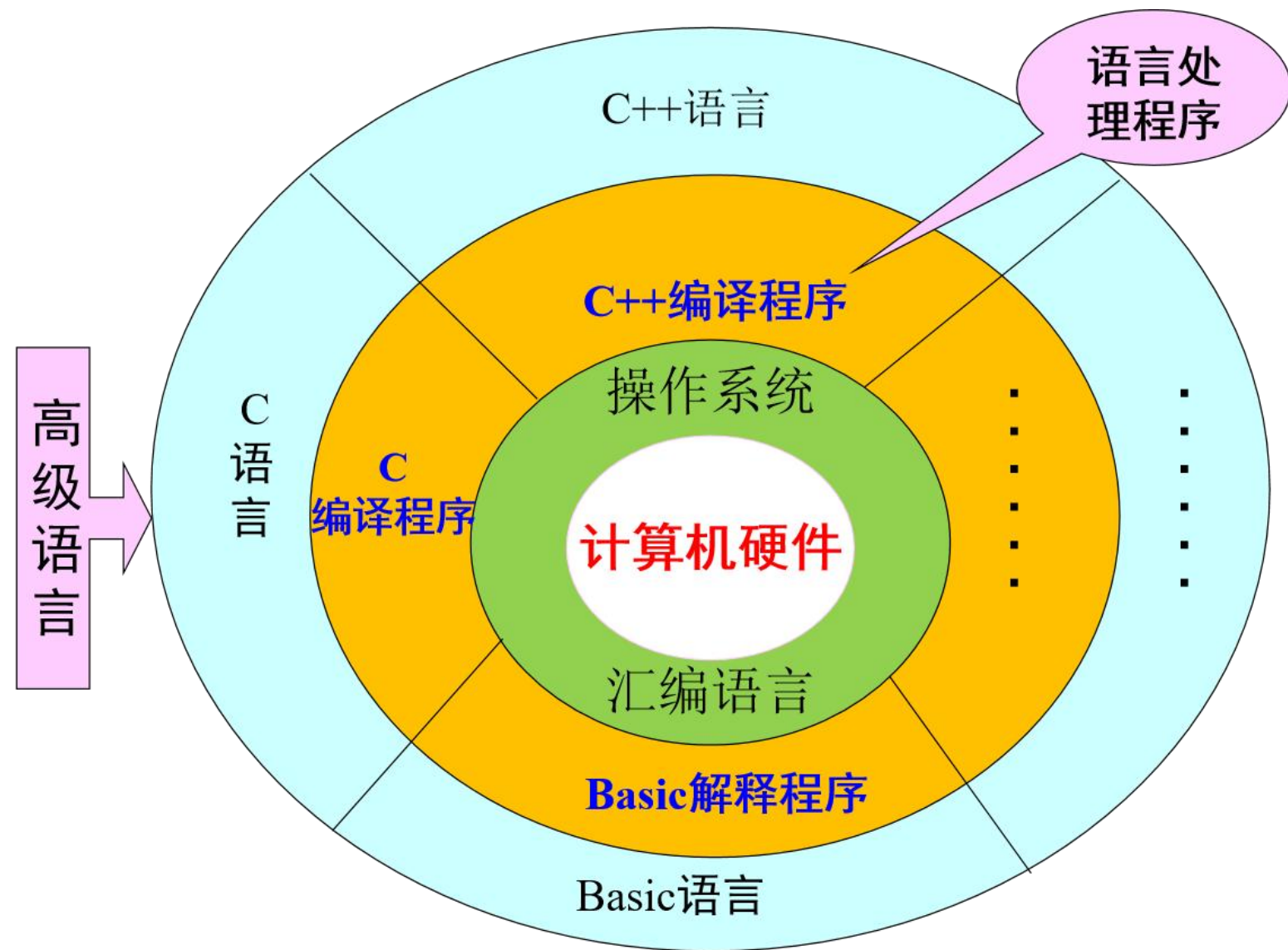
```
assume cs:code, ds:data
data segment
    dw 1234h,5678h
data ends
code segment
start:  mov ax, data
        mov ds, ax
        mov ax, ds:[0]
        mov bx, ds:[2]
        mov cx, 0
        add cx, ax
        add cx, bx
        mov cx, ds:[4]
        mov ax, 4c00h
        int 21h
end start
code ends
```

■ 高级语言(High Level Language)

- 其表示方法更接近于待解问题的表示方法
- 定义数据、描述运算、控制流程、传输数据
- 如：C、FORTRAN、PASCAL、C++、JAVA、SQL语言

```
int main
{
    int a,b,c;
    a=1234h;
    b=5678h;
    c=a+b;
    return 0;
}
```

2、编译程序所处的层次



什么叫翻译程序

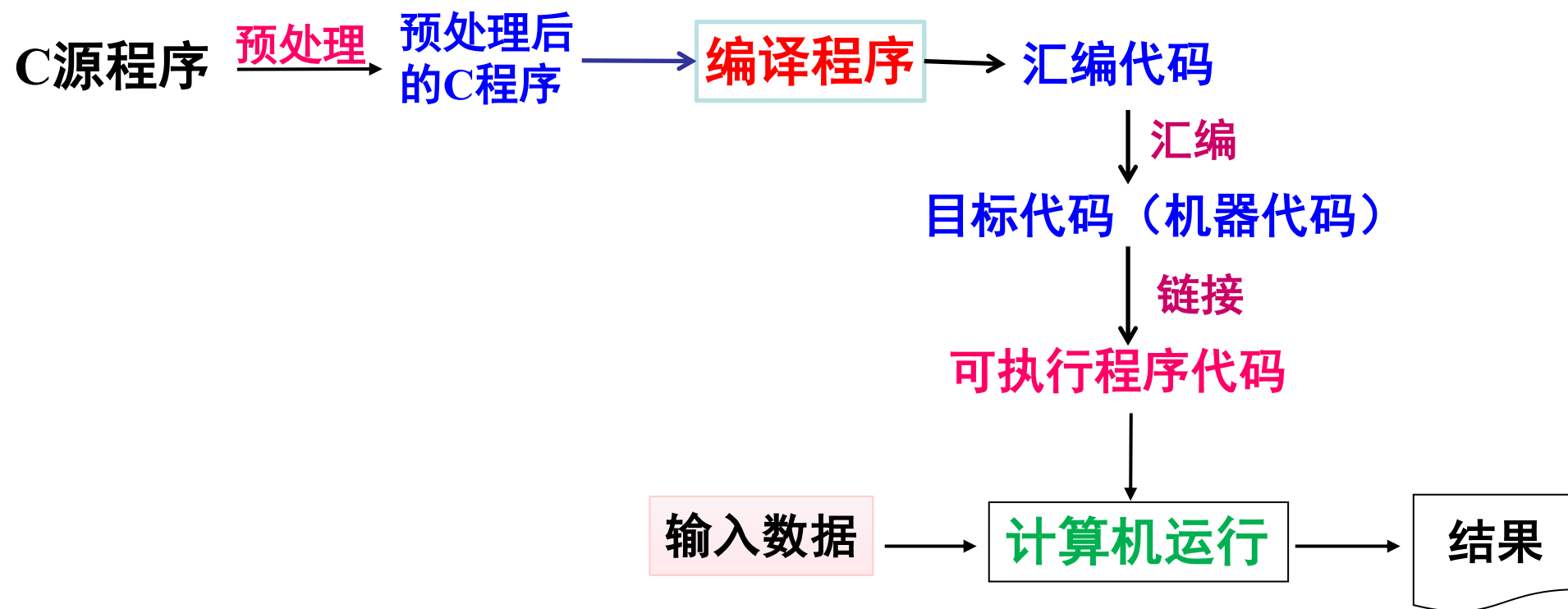
- **翻译程序**：能够将用某种编程语言编写的程序转换成另一种编程语言的程序，而且后者与前者保持**逻辑上是等价的**。

例如：



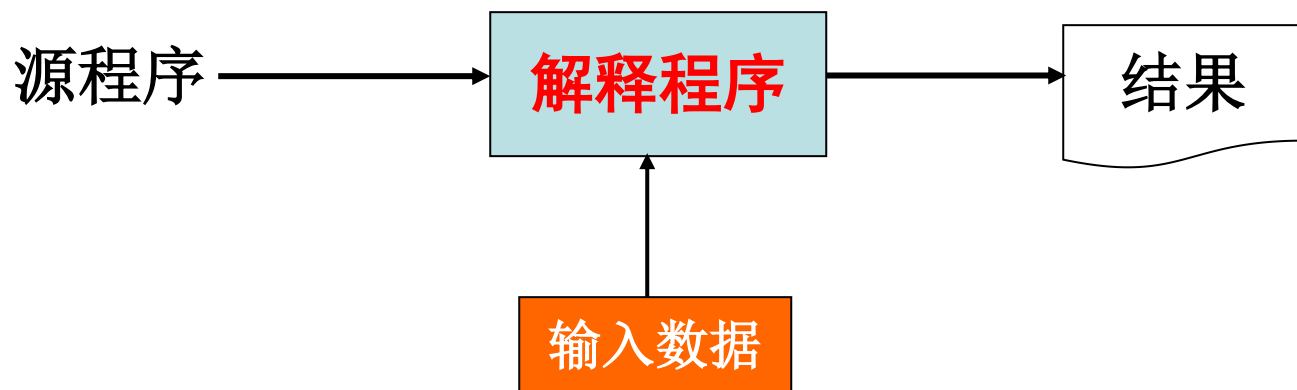
什么是编译程序

- **编译程序(Compiler)**: 将高级程序设计语言程序翻译成**逻辑上等价**的**低级语言(汇编语言,机器语言)**程序的翻译程序。

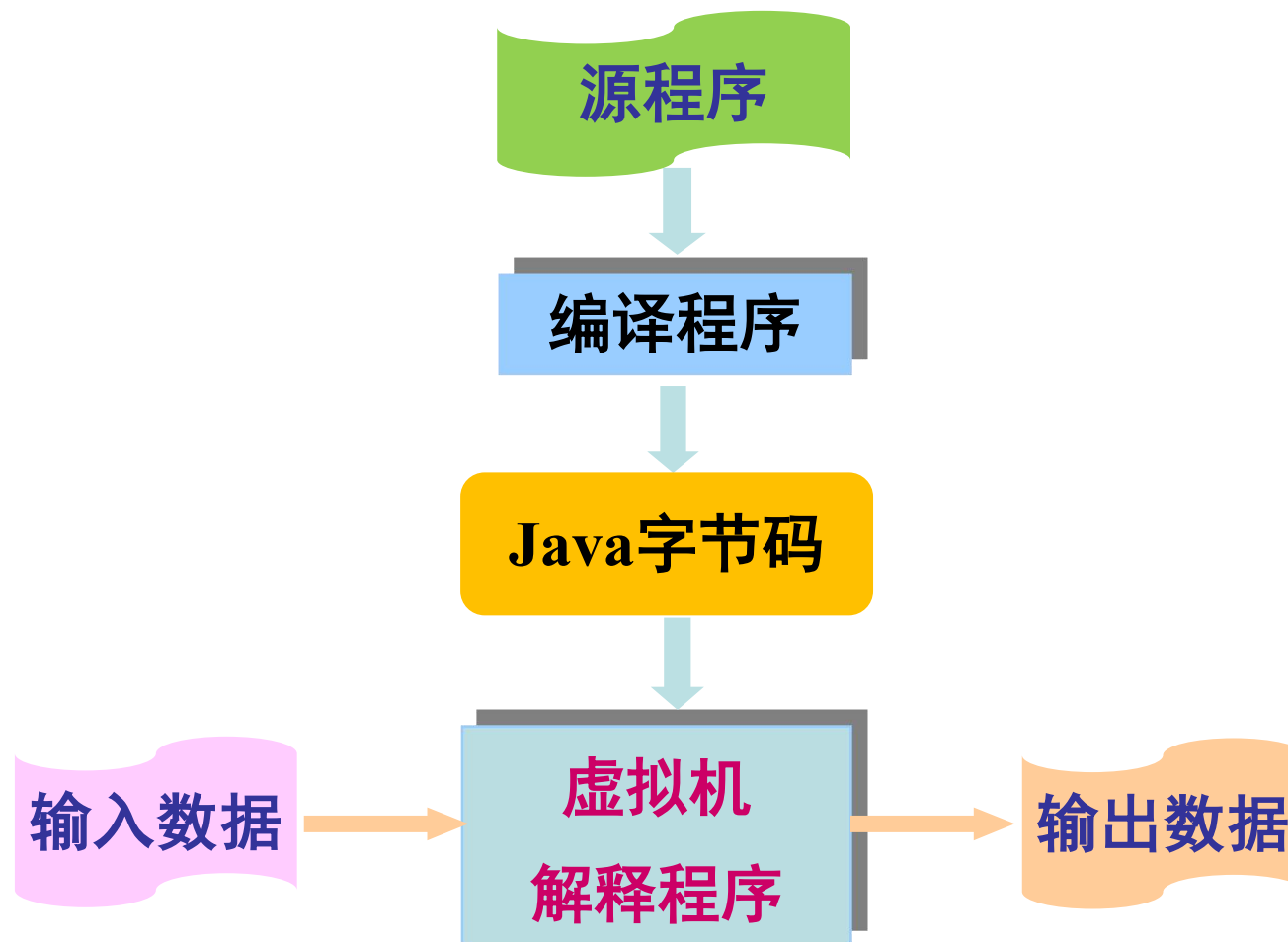


什么是解释程序

- **解释程序(Interpreter)**: 将高级程序设计语言写的源程序作为输入，**边解释边执行源程序本身**，而**不产生目标程序**的翻译程序。



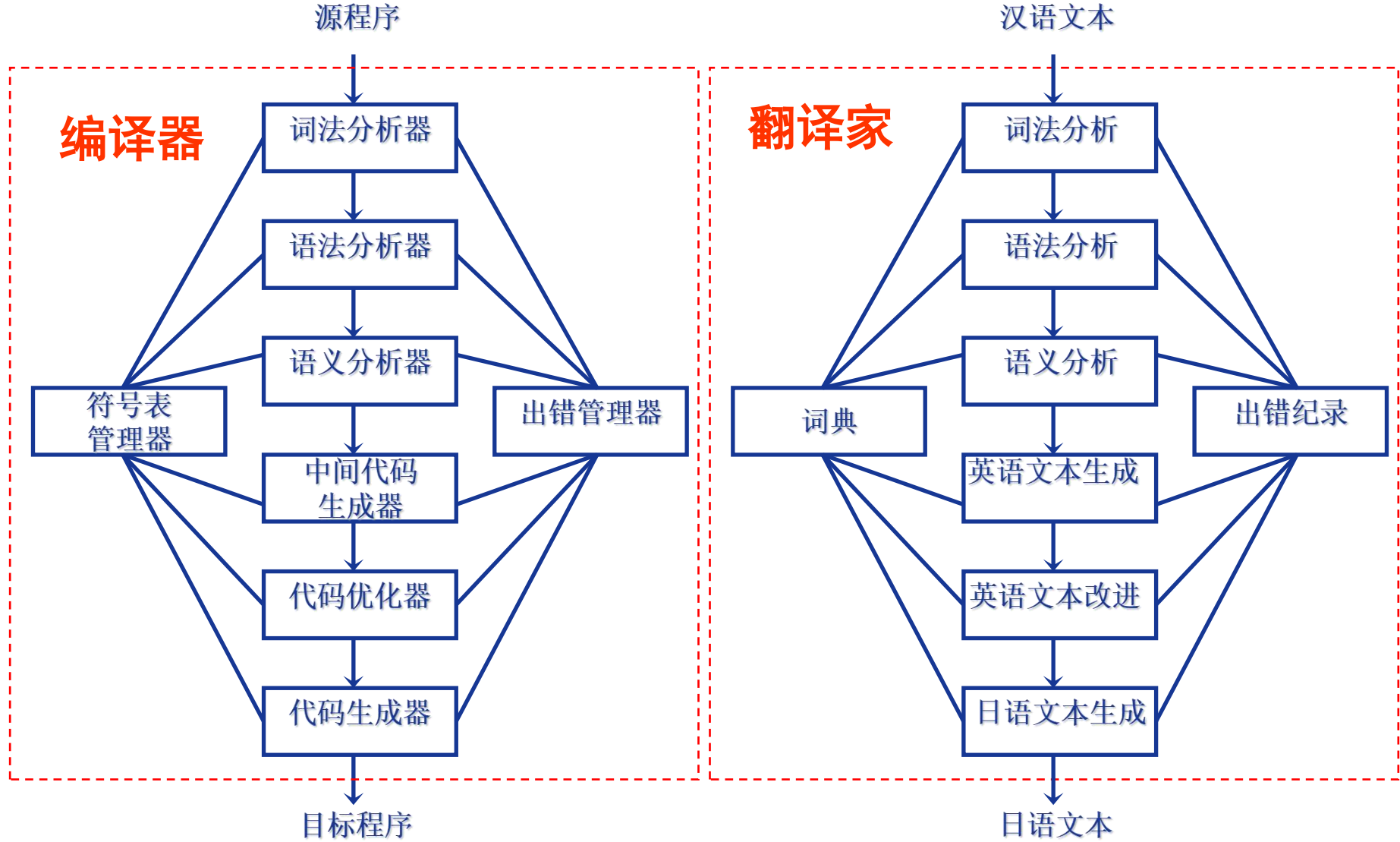
Java语言的编译-解释混合编译器



编译器功能

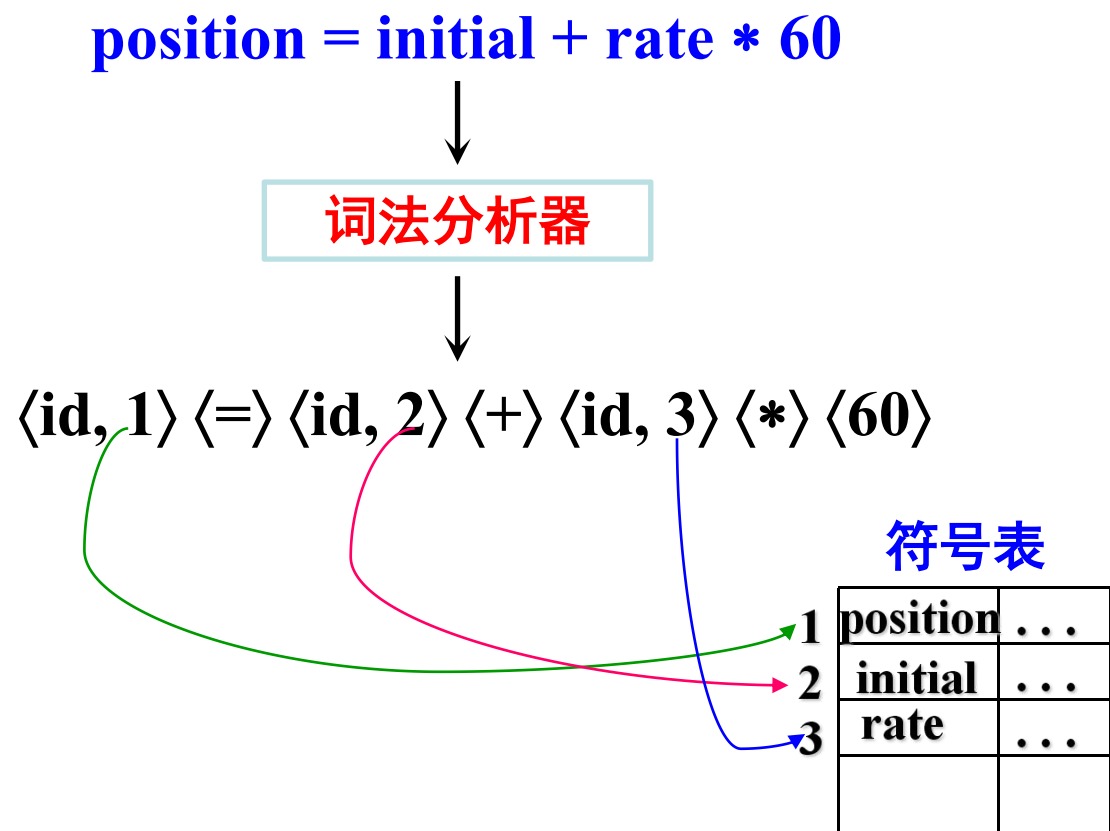
- 它读入用某种语言编写的源程序，并翻译成一个与之等价的目标语言编写的源程序。完成从源语言到目标语言的转换
 - ✓ 源语言：通常是高级语言（C,Java,...）
 - ✓ 目标语言：汇编语言，或者其他形式的低级语言（如Java字节码）
- 编译器实现技术已经发展成熟，并且划分为多个功能相对明确的模块

4、编译器的基本结构



词法分析

- 词法分析将源代码分解成**词法单元(token)**，并识别每个**词法单元的类型**。



请思考下列问题：

- 如何告诉词法分析器，什么样程序片段是正确的词法单元？
- 如何设计词法分析器，使它能够正确识别用该语言编写的程序中的词法单元？

符号表(Symbol Table)

- 符号表是编译器的重要数据结构，用于存储程序中出现的标识符（如变量名、函数名等）及其相关信息。符号表在编译过程中起着记录和管理符号信息的作用，以便编译器在后续的各个阶段使用。
- 符号表由多个符号表条目组成，每个符号表条目对应一个标识符，包含该标识符的信息。符号表通常包括标识符名称、数据类型、作用域、存储位置等字段。
- 为了快速查找符号表中的符号信息，符号表通常使用哈希表来实现。哈希表能够根据标识符名称快速定位到对应的符号表条目，提高符号查找的效率。
- 通常编程语言中的关键字单独建立符号表。

符号表的示例

假设有以下简单的代码段：

```
int main() {  
    int a = 5;  
    float b = 3.14;  
    return 0;  
}
```

标识符名称	数据类型	作用域	其他信息	
main	int()	global	函数	
a	int	main	变量	
b	float	main	变量	

示例展示了一个编译器符号表的基本结构，包括标识符名称、数据类型、作用域等信息。实际的符号表会更加复杂，包括更多的字段和更多的标识符信息，用于支持编译器在编译过程中的各种分析和优化操作。

语法分析

- 语法分析是编译过程中的重要步骤，其主要功能是根据源代码的词法单元(token)流构建语法树（Parse Tree）或抽象语法树（Abstract Syntax Tree, AST）。

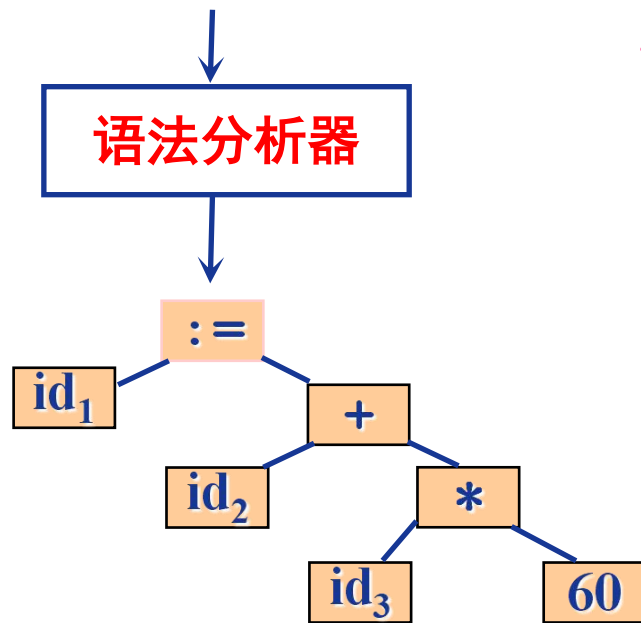
$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$

请思考下列问题：

1. 如何告诉语法分析器，该语言能够表达哪些正确语法结构？
2. 如何设计语法分析器（算法），使它能够正确识别用该语言编写的程序中的语法结构？

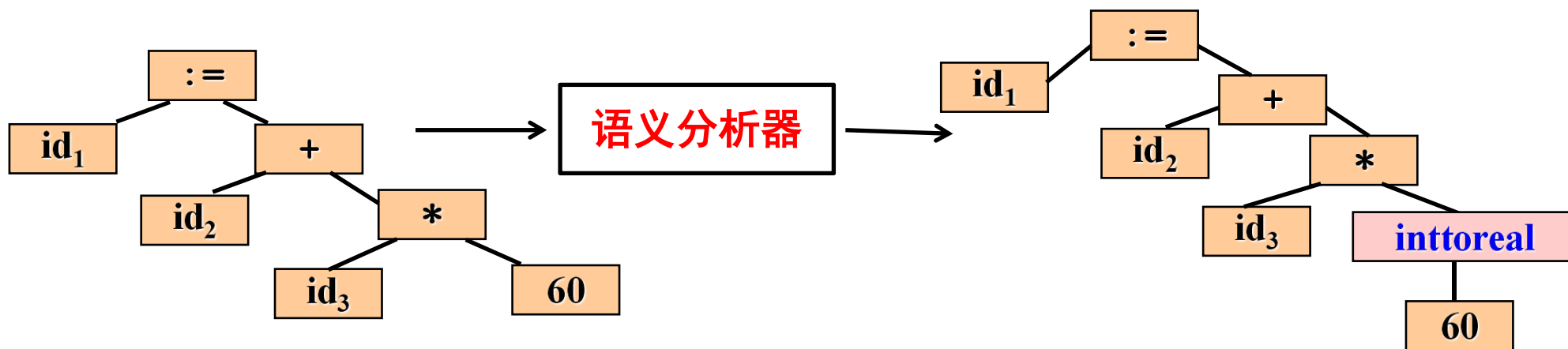
符号表

1	position	...
2	initial	...
3	rate	...

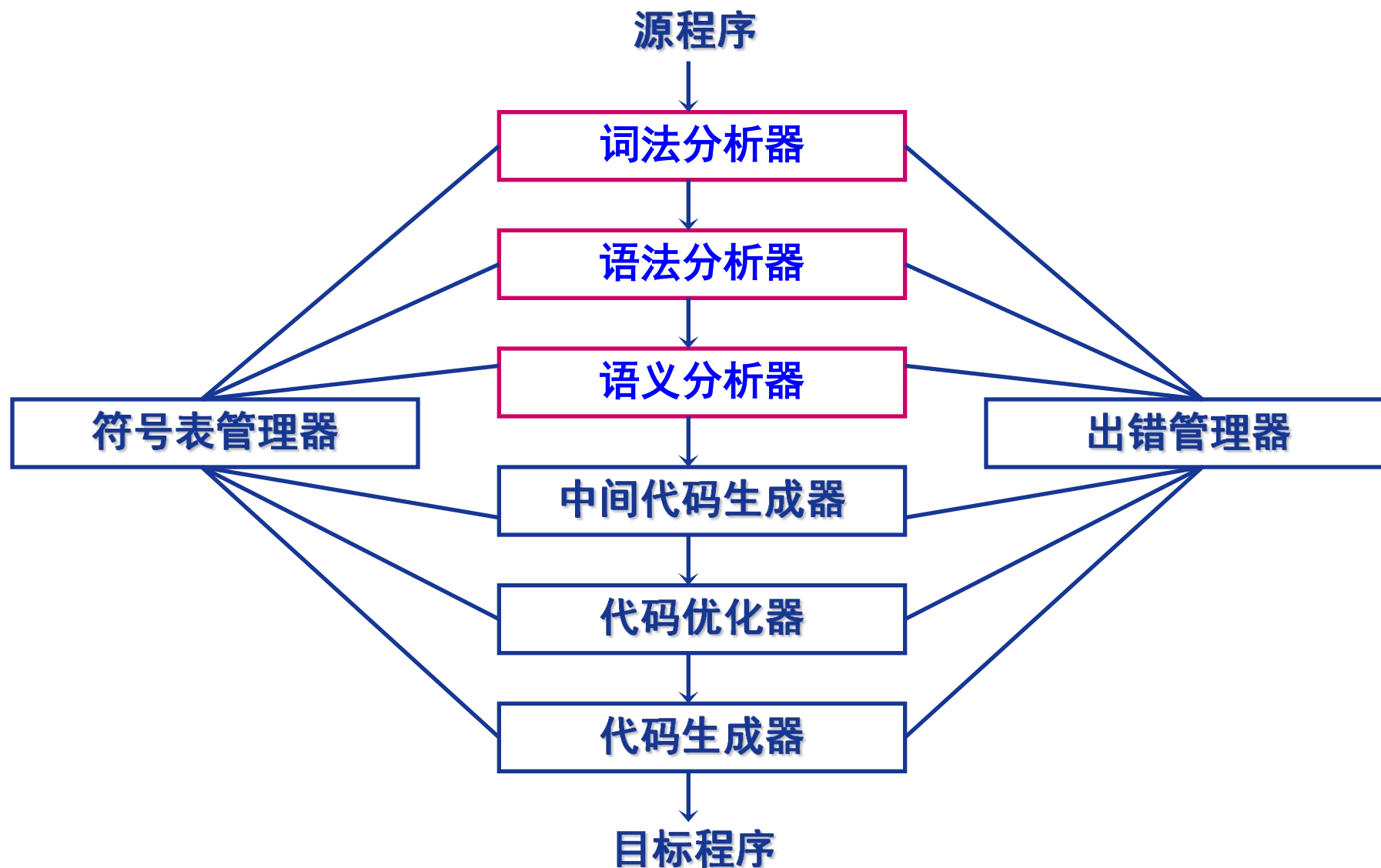


语义分析

- 语义分析的主要任务是对源代码的语义进行分析和检查。语义分析器会根据语法树或抽象语法树进行语义检查，确保程序的语义是正确的。包括变量的作用域、类型检查、函数调用等。
- 类型检查：确保变量的使用和赋值符合类型规定。如果源代码中存在类型不匹配的情况，语义分析器会报告类型错误。
- 作用域分析：确保变量在正确的作用域内被引用和赋值。如果源代码中存在作用域问题，如变量未声明或重复声明等，语义分析器会进行检查并报告错误。
- 语义错误检测：语义分析器会检测源代码中的语义错误，如未定义的变量、不兼容的类型、不合法的操作等，并生成相应的错误消息。

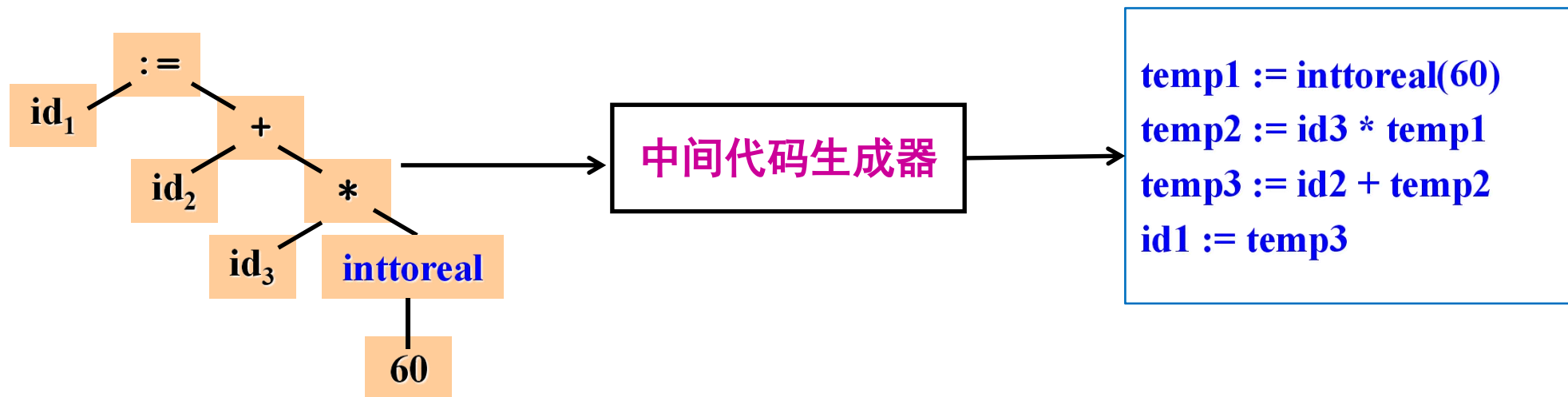


前三个阶段完成对源程序的分析



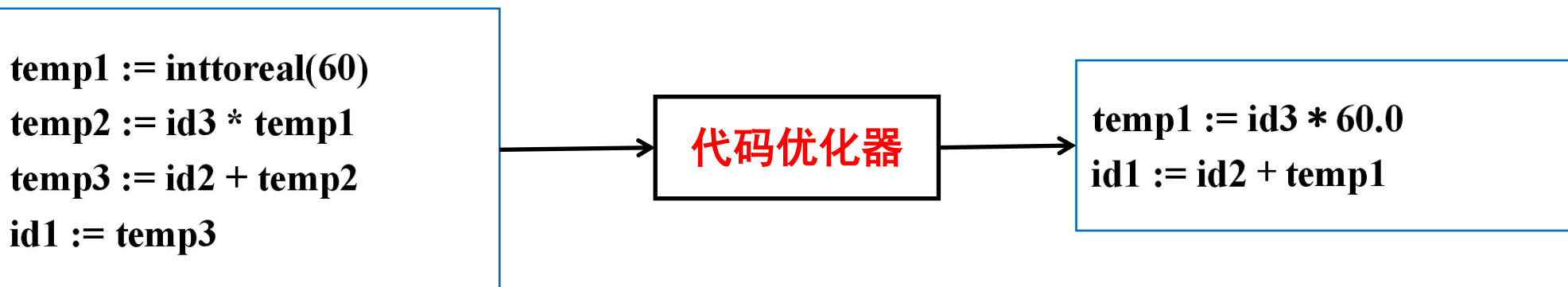
中间代码生成

- 中间代码生成的目标是将源代码转换为一种**中间表示形式**，通常是一种**抽象的、与具体硬件无关的表示形式**（如**三地址码、四地址码**）。中间代码是一种介于源代码和目标代码之间的表示形式，**便于后续优化和代码生成**。



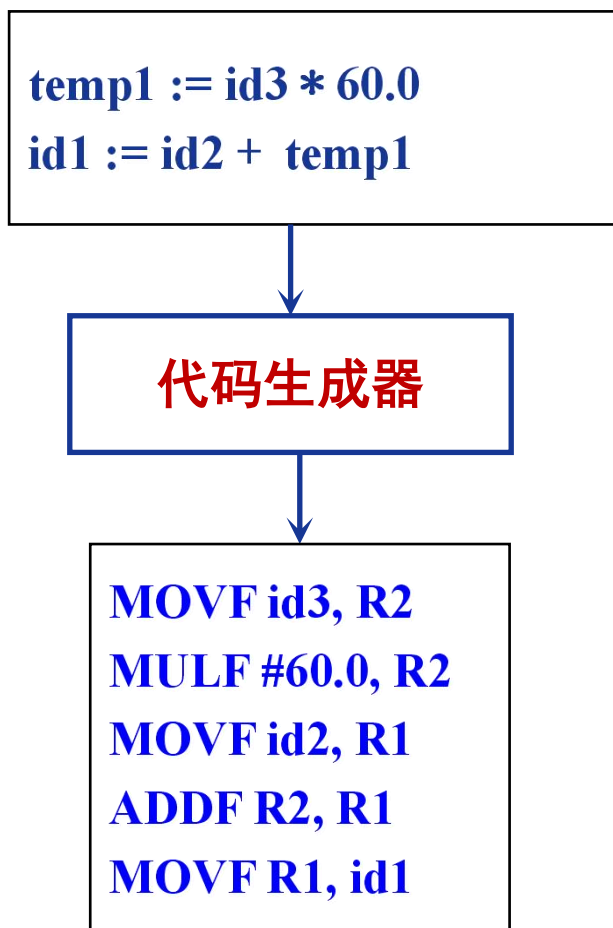
代码优化

- 代码优化的目标是通过改进程序的性能、减少资源消耗或者改善代码质量来提高程序的效率。代码优化可以在中间代码生成阶段或目标代码生成阶段进行。
- 代码优化涉及多种技术，包括但不限于死代码消除、循环优化、数据流分析、指令调度等。旨在改进程序的执行效率、减少资源消耗和提高代码质量。

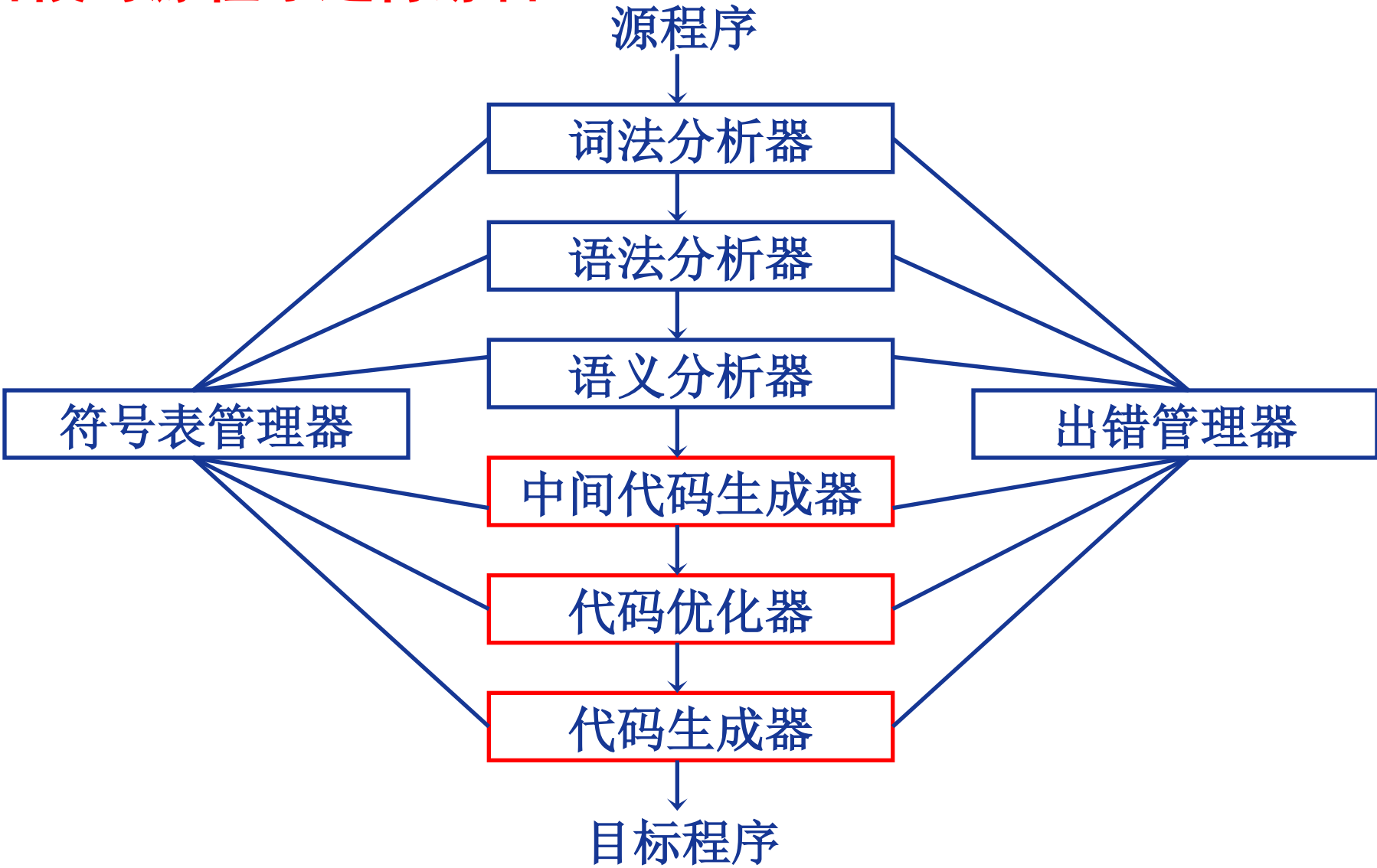


目标代码生成

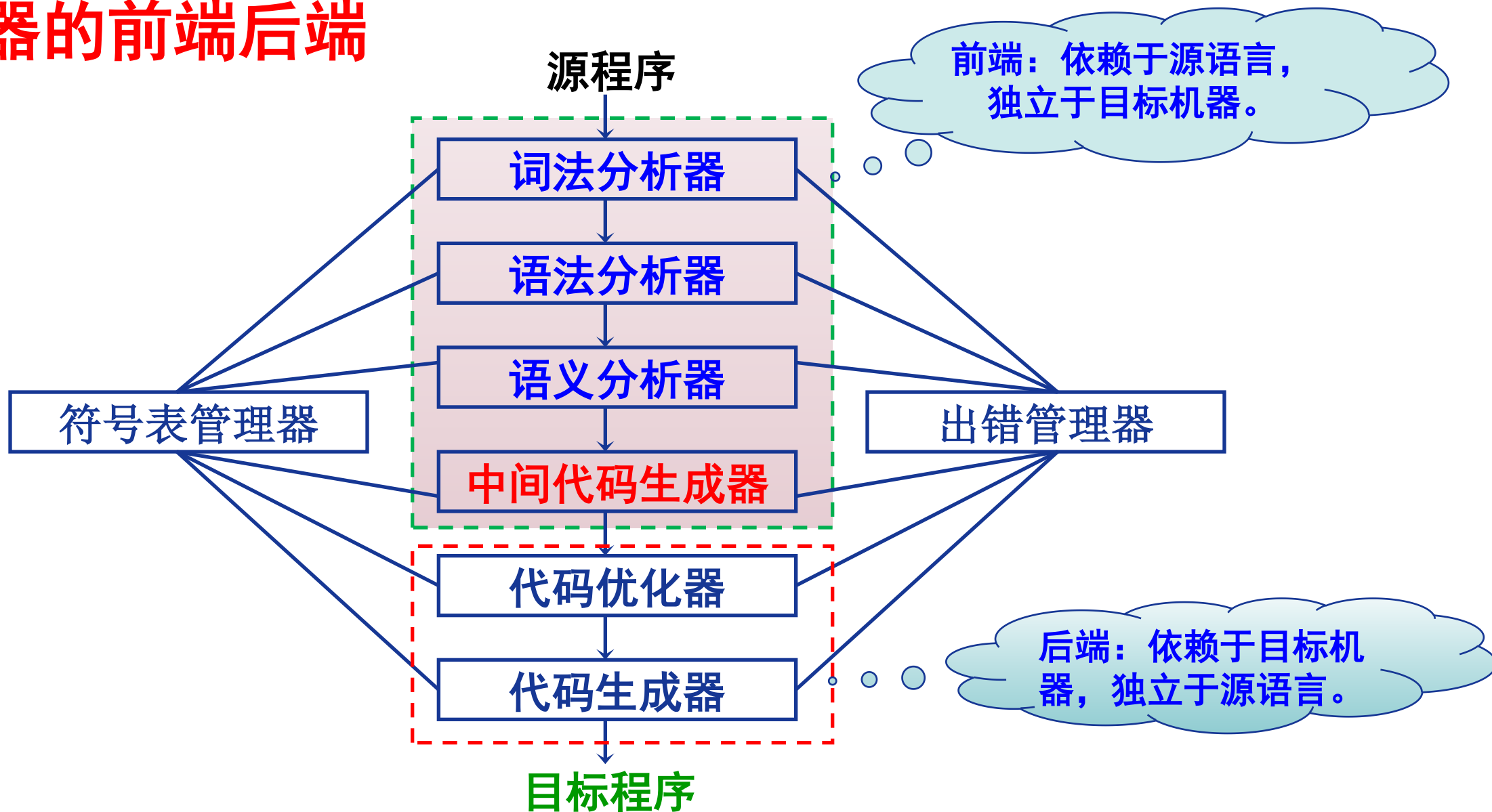
- 输入中间代码，映射为**目标语言代码**，如果为机器代码，必须为**程序变量选择寄存器或内存地址**



后三个阶段对源程序进行综合



编译器的前端后端

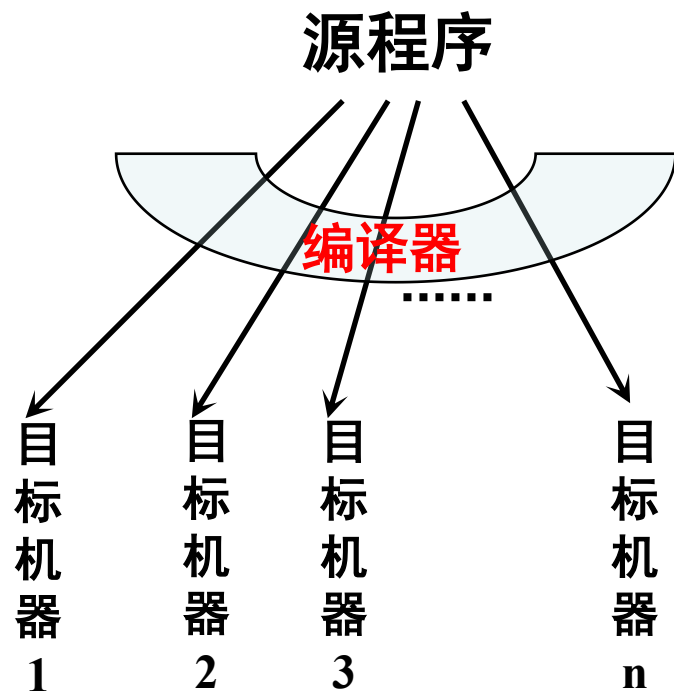


编译器划分为前端和后端的优点

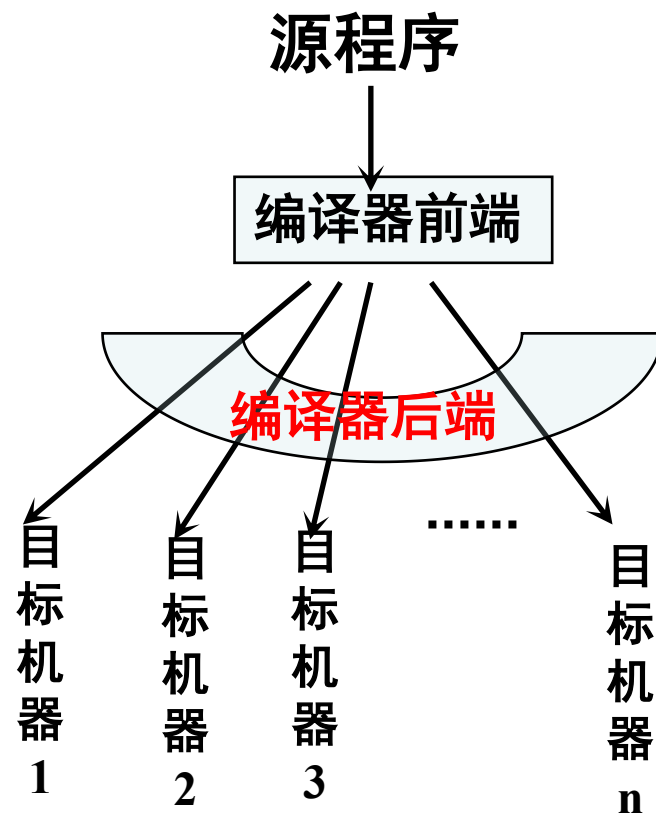
- 把编译过程分成前端和后端两部分
- **前端**：只依赖于源程序，独立于目标机器（生成中间代码）
- **后端**：依赖于目标机器，与源程序无关，只与中间语言有关（从中间代码生成目标代码）
- **好处**：提高开发编译器的效率
 - ① 取一个编译器的前端，重写它的后端以产生同一源语言在另一机器上的编译器
 - ② 不同的前端使用同一个后端，从而得到一个机器上的几个编译器（采用同一中间语言）

编译器划分为前端和后端的优点

不区分前端和后端的编译器



区分前端和后端的编译器



AI 编译器简介

将深度学习模型描述自动翻译并优化为底层硬件可执行代码的工具链

- ① 输入：PyTorch、TensorFlow、MindSpore等框架构建的模型
- ② 输出：GPU、NPU、DSP、FPGA等硬件上的高效机器码
- ③ 核心目标：在语义不变的前提下，最大化硬件利用率、降低延迟和内存占用
- ④ 本质：连接“AI算法”与“异构硬件”的桥梁

为什么需要AI编译器

- ① 硬件多样性：不同芯片指令集、缓存结构、并行方式差异巨大
- ② 框架碎片化：多框架、多硬件组合导致开发维护成本指数级增长
- ③ 模型复杂度暴涨：百亿/千亿参数大模型需要全局优化精力的支撑
- ④ 开发效率与性能解耦：算法工程师专注模型，编译器自动优化底层实现

AI编译器 vs 传统编译器

维度	传统编译器	AI编译器
输入	C/C++等高级语言	计算图/图IR
核心操作	标量、指针、控制流	张量运算
优化目标	指令选择、寄存器分配、循环优化	算子融合、内存复用、布局转换、并行调度
主要挑战	别名分析、复杂控制流	动态形状、异构设备、分布式策略
运行模式	静态编译	静态编译
典型代表	GCC、LLVM	TVM、XLA、MLIR

AI编译器整体架构

三个关键层次：

- ① 前端（Frontend）：解析多种框架模型，生成统一IR
- ② 中间层（Optimizer）：执行计算图级优化
- ③ 后端（Backend）：面向具体硬件生成代码

配套基础设施：

- ① 多层IR（Graph IR / Tensor IR / LLVM IR）
- ② 自动调优模块
- ③ 运行时与调度器

前端：框架到计算图

输入：ONNX、TorchScript、SavedModel等模型格式

主要工作：

- ① 算子解析与映射（高层API → 编译器原语）
- ② 动态形状处理（符号化或特化）
- ③ 控制流转换（Python if/for → 计算图控制流节点）

输出：统一的计算图表示

中间层：图优化（Graph Optimizer）

算子融合： Conv+BN+ReLU $\rightarrow$ 一个融合算子，减少中间数据读写

常量折叠： 编译期计算可确定的常量

代数化简： $1 * x \rightarrow x$ ，消除冗余操作

内存规划： 缓冲区复用、内存池化

布局优化： NCHW $\rightarrow$ NHWC等，减少转置开销

并行调度： 分析依赖关系，挖掘并行度

后端：代码生成与硬件适配

- ① 算子库选择：优先调用cuBLAS、cuDNN、oneDNN等高性能库
- ② 自动调优（Auto-tuning）：搜索分块大小、循环顺序、向量化程度
- ③ 内存层次优化：合理使用寄存器、共享内存、全局内存
- ④ 代码生成：从Tensor IR生成CUDA/OpenCL/ISP C等
- ⑤ 最终通过LLVM编译到目标指令集

多层IR设计

- ① Graph IR（高层）：表达整个计算图，用于跨算子优化
- ② Tensor IR（中层）：描述单算子/融合组的循环和访存
- ③ 后端IR（低层）：如LLVM IR，用于最终指令生成

MLIR的价值：

- ① 提供可扩展的多级Dialect
- ② 解决编译器碎片化问题
- ③ 统一不同抽象级别的优化

关键技术（1）——自动微分与反向训练

- 支持训练场景，需构建反向计算图
- 静态图自动微分：图重写生成梯度运算
- 附加优化：
 - ① 梯度累积
 - ② 梯度裁剪
 - ③ 混合精度训练
- 与动态图框架（PyTorch）的JIT编译结合

关键技术（2）——动态形状与JIT编译

- 动态场景：变长batch、NLP变长序列
- 解决方案：
 - ① 符号化形状推理
 - ② 运行时按需编译（JIT）
 - ③ 编译结果缓存复用
- 示例：PyTorch Inductor，对频繁形状动态生成优化内核

关键技术（3）——自动调优（Auto-tuning）

- 为什么需要：手写算子模板无法覆盖所有硬件与形状
- 方法：基于模板的参数搜索
 - ① 贝叶斯优化 / 遗传算法
 - ② 机器学习代价模型预测
 - ③ 代表系统：TVM Ansor、Halide Autoscheduler
- 目标：为每个算子寻找最优实现变体

分布式编译

- 面向大规模训练，编译器自动完成：
 - ① 计算图切分
 - ② 插入通信原语（AllReduce、AllGather等）
 - ③ 并行策略搜索（数据并行、模型并行、流水线并行、张量并行）
- 代表工作：
 - ① Google GSPMD
 - ② Alpa（自动并行编译器）
- 目标：用户无需手工指定并行策略，一条命令完成多卡部署

主流AI编译器生态

编译器	所属/开源	特点
XLA	Google	支持TF/JAX，HLO IR，TPU优化
TVM	Apache	端到端多硬件支持，强Auto-tuning
MLIR	LLVM项目	多层IR基础设施，可构建自定义编译器
Triton	OpenAI	GPU内核编写语言，类似NumPy
MindSpore AKG	华为	自动算子和图执行优化
IREE	Google	基于MLIR，侧重推理部署
OneCompiler	OneFlow	结合性能与分布式策略

面临的挑战

- ① 硬件演进速度快，编译优化难以跟上
- ② 动态、稀疏模型（MoE、动态长度）难以静态优化
- ③ 多目标冲突：延迟、内存、精度、吞吐量
- ④ 编译时间开销大，影响迭代效率
- ⑤ 可调试性差：优化后难以溯源定位问题
- ⑥ 大模型内存压力：激活值爆炸，需重计算/卸载

未来趋势

- ① MLIR生态统一：构建更通用的多层级编译基础设施
- ② 编译与运行时协同：在线重编译、动态调度
- ③ 端云联动：同编译器支持云端训练与终端部署
- ④ 大模型专用优化：KV Cache管理、MoE负载均衡、长序列优化
- ⑤ 自动化并行策略：端到端自动分布式编译
- ⑥ AI辅助编译器：用机器学习指导编译优化决策

本章作业

1. 简要解释编译器的作用以及编译过程的各个阶段。
2. 解释编译器前端和后端的区别，并列举各自的主要任务。
3. 什么是语法分析？列举一些常见的语法分析方法。
4. 解释词法分析的作用，并说明词法分析器的输入和输出。
5. 什么是中间代码？列举几种常见的中间表示形式。